SENG 319: Software Design Patterns – Group Assignment 3 Report

1. Introduction

This report talks about the technical debt in the CheckStyle project. We used SonarQube to find problems in the code. We look at problems like Code Smells, Bugs, Security Issues, and Test Coverage. We explain the effect of these problems on maintainability, reliability, and development speed. Also, we give advice on fixing the problems and how to prevent them in the future.

2. Technical Debt Summary

2.1 Core Quality Metrics and Technical Debt Measurements

Metric	Value	Rating	Analysis & Implication
Total Technical Debt Effort (Remediation Effort)	8 Days	N/A	162 problems can be fixed in a short time. Debt is small but still important.
Debt Ratio	≤5%	A	Debt is very small compared to overall development.
Total Issues Found	162	N/A	All Bugs, Vulnerabilities, and Code Smells counted.
Maintainability	144 Open Code Smells	A	High number of code smells can slow future work.
Security	2 Open Issues	D	1 High severity issue makes security critical.
Reliability (Bugs)	16 Open Issues	C	Medium bugs may cause crashes or errors.
Test Coverage	0.0%	E	No tests exist; refactoring is risky.
Duplication	1.7%	A	Code is mostly well-structured with low duplication.

Analysis: Overall maintainability and debt ratio are okay, but security and test coverage are critical problems.

2.2 Severity and Category Distribution

Severity Level	Instance Count	Impact & Priority
Blocker	1	Very high priority, can stop development.
High	28	Serious security or reliability problems.
Medium	63	Increases maintenance cost; code smells.
Low	52	Minor issues, mostly style or readability.
Total	162	All issues combined (Bugs, Vulnerabilities, Code Smells).

2.3 Primary Causes of Technical Debt

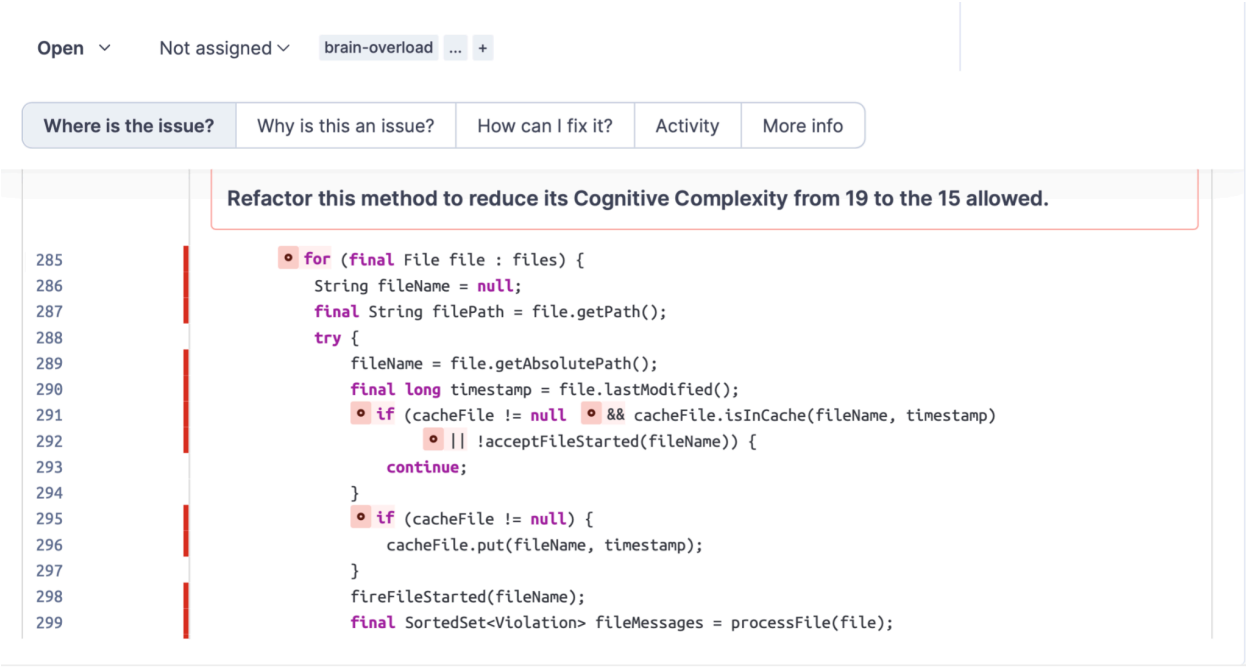
High-Impact Issue Type	Example	Cause
High Cognitive Complexity	Refactor this method to reduce complexity	Methods are too complex with many loops and conditions. Breaks SRP.
God Class	Split this "Monster Class"	One class does too much, hard to maintain or test.
Security Vulnerability	High severity issue	No strict security checks during development.
Missing Test Coverage	0.0%	No tests, risky refactoring, code smells accumulate.

3. Impact Analysis

3.1 High-Impact Debt Analysis: Risks

High-Impact Debt Category	SonarQube Finding	Effect on Maintainability, Reliability, Speed	Justification
Process Debt	Test Coverage 0.0%	Hard to refactor code safely. Every change needs manual testing.	Critical: prevents safe fixing of code smells.
Architectural Debt	Cognitive Complexity 19/15	Hard to read and understand methods. High chance of errors.	Deep nested ifs and loops; violates SRP.
Security & Stability	1 Blocker, 2 High Issues	Can lead to data loss, unauthorized access. System unstable.	Must fix immediately; emergency work stops other features.

Code Snippet: Cognitive Complexity Example



The complexity source is clearly visible in the following code, illustrating the violation of SRP and poor structural design:

```
// CheckStyle - Checker.java (L285 ff., Complexity +19)
for (final File file : files) { // +1
    // ...
    try {
// ...
        if (cacheFile != null && !cacheFile.isInCache(fileName, timestamp)) { // +1
            if (!acceptFilesStarted(fileName)) { // +2 (Nested if)
                continue;
            }
        }
        if (cacheFile != null) { // +1
            cacheFile.put(fileName, timestamp);
        }
        // ...
    }
```

3.2 Low-Impact Debt Analysis: Minor Issues

Low-Impact Debt Category	SonarQube Finding	Effect	Justification
Style & Readability	52 Low Issues	Code is less readable, minor impact	Can fix during normal work.
Micro-Performance	Minor warnings	Small effect on performance	Quick fixes, no urgent risk.

4. Refactoring Recommendations

4.1 Preventing Future Technical Debt

Guideline	Reason	Implementation
Mandatory Test Coverage	0.0% coverage is risky	Set SonarQube Quality Gate: minimum 80% coverage before PR merge.
Strict Complexity Threshold	High complexity raises maintenance cost	Reduce Cognitive Complexity threshold to 10–12. Use small focused methods.
Security-First Policy	Blocker/high issues are critical	Fail build if any critical issue exists. Fix before merging.

4.2 Servicing Current Issues

High-Impact Issue	Refactoring Technique	Steps
Lack of Test Coverage 0.0%	Characterization Tests	1. Write tests for existing methods. 2. Reach 80% coverage for high-complexity methods.
Cognitive Complexity	Extract Method & Decompose Conditional	1. Break complex ifs into small methods. 2. Replace logic with clear, single-purpose calls.
God Class	Extract Class & Dependency Injection	1. Separate responsibilities (Cache, File, Rules). 2. Create new classes (CacheManager, FileHandler). 3. Inject new classes in Checker.

5. Discussion & Reflection

5.1 As a Developer

- Follow SRP and SOLID rules.
- Refactor complex methods immediately (Extract Method).
- Apply "Boy Scout Rule": leave code cleaner than before.
- Use TDD or Characterization Tests for safety before refactoring.

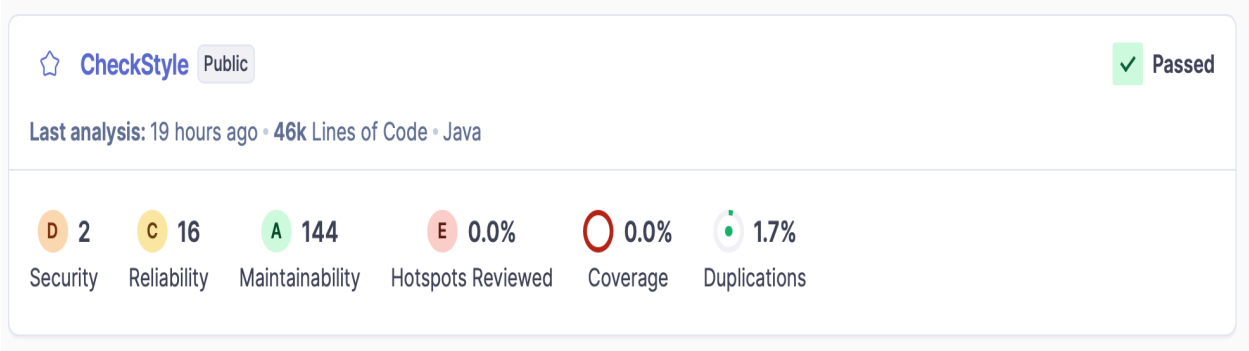
5.2 As a Project Manager

- Make technical debt visible and budgeted.
- Allocate 15–20% of sprint time for debt repayment.
- Use SonarQube Quality Gate: zero tolerance for critical issues.
- Explain debt in business terms: e.g., "God Class slows feature delivery by 40%."

6. Conclusion

The CheckStyle project has mostly small debt but critical issues in security and test coverage. High cognitive complexity and God Classes slow development and increase risk. Fixing these issues first and preventing future debt with strict testing, complexity rules, and security policies is essential. Both developers and managers must work together to maintain code quality and avoid future debt.

7.Appendices



Severity

Blocker	1
High	28
Medium	63
Low	52
Info	18

Reliability

A	≥ 0 info issues	0
B	≥ 1 low issue	0
C	≥ 1 medium issue	1
D	≥ 1 high issue	0
E	≥ 1 blocker issue	0

Security Review

The percentage of reviewed (fixed or safe) security hotspots

A	≥ 80%	0
B	≥ 70% to <80%	0
C	≥ 50% to <70%	0
D	≥ 30% to <50%	0
E	< 30%	1

Coverage

≥ 80%	0
70% - 80%	0
50% - 70%	0
30% - 50%	0
< 30%	1
No data	0

Size

XS < 1k	0
S 1k - 10k	0
M 10k - 100k	1
L 100k - 500k	0
XL > 500k	0

Duplications

< 3%	1
3% - 5%	0
5% - 10%	0
10% - 20%	0
> 20%	0
No data	0

Security Review

The percentage of reviewed (fixed or safe) security hotspots

A ≥ 80%	0
B ≥ 70% to <80%	0
C ≥ 50% to <70%	0
D ≥ 30% to <50%	0
E < 30%	1

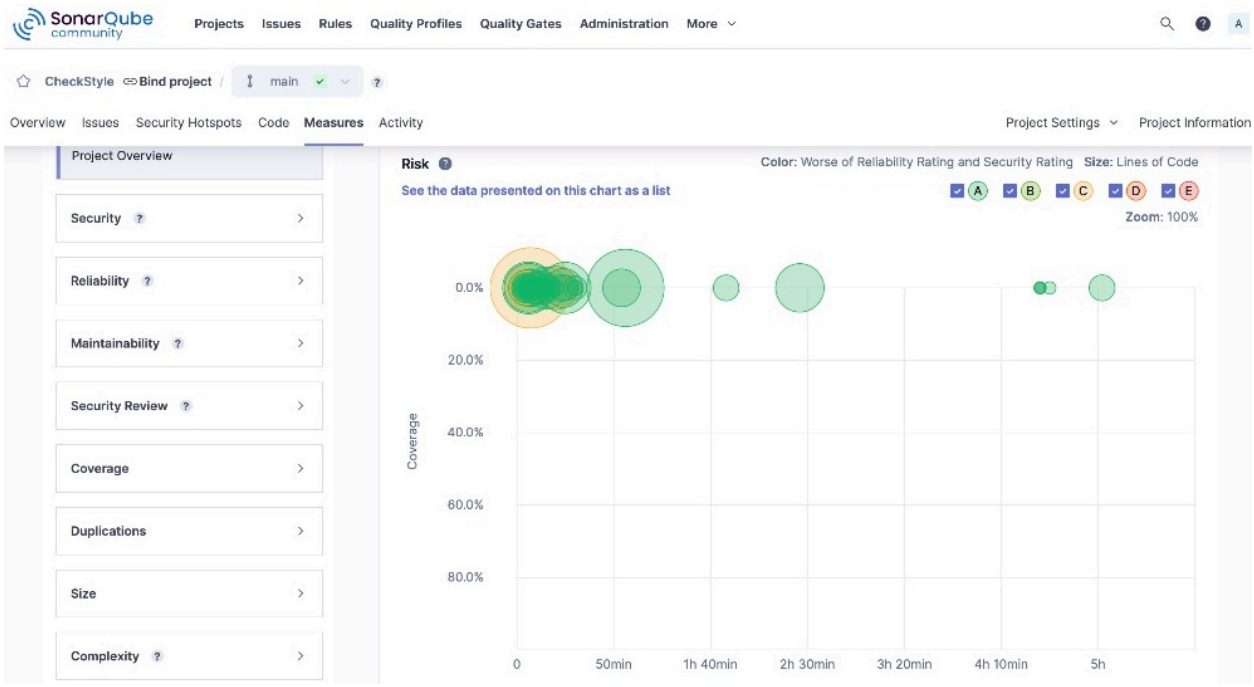
Maintainability

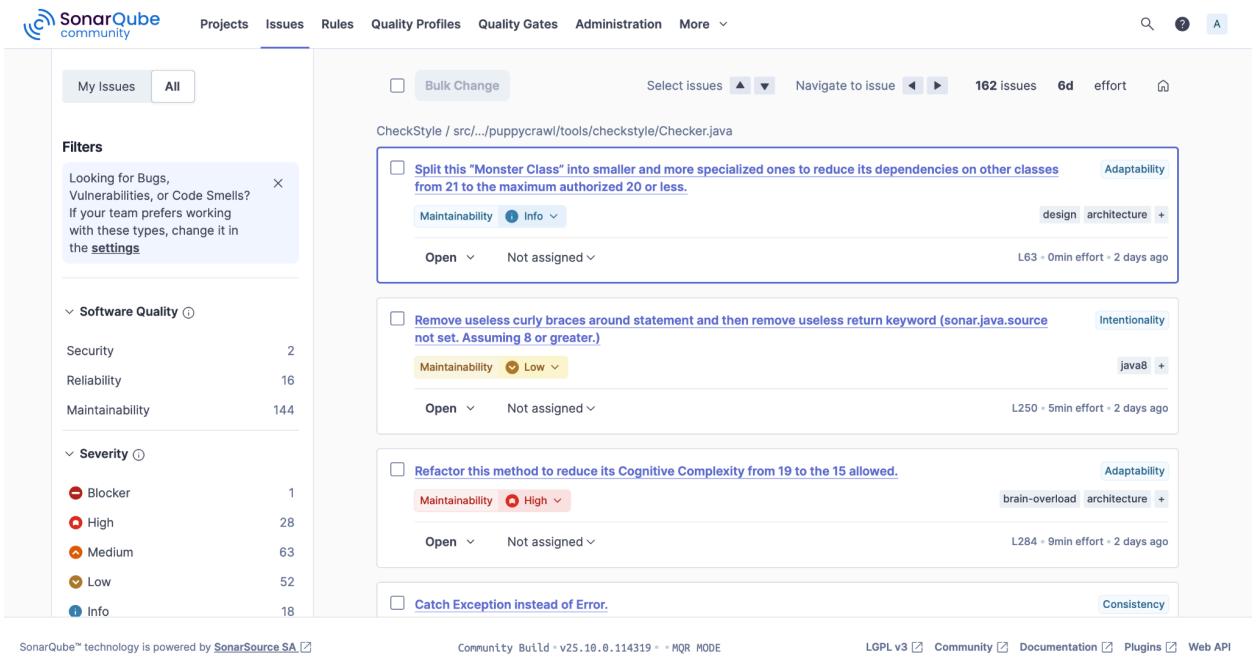
Ratio of the estimated time needed to fix all outstanding maintainability issues to the size of the project

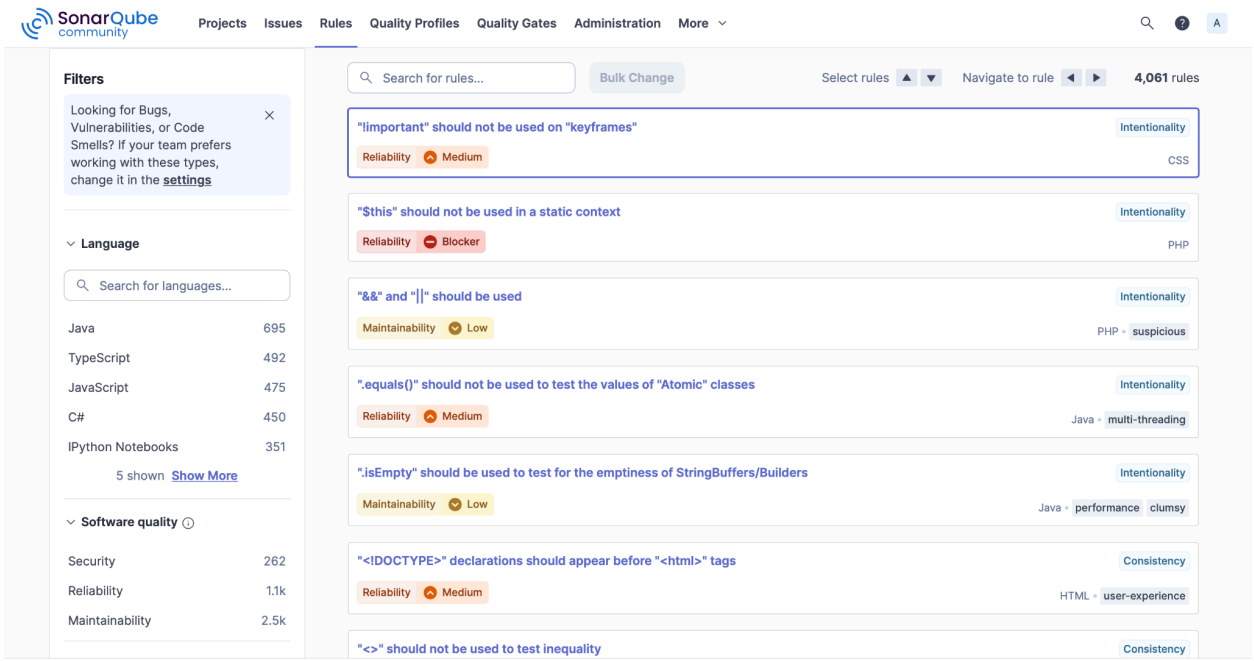
A	≤ 5% to 0%	1
B	≥ 5% to <10%	0
C	≥ 10% to <20%	0
D	≥ 20% to <50%	0
E	≥ 50%	0

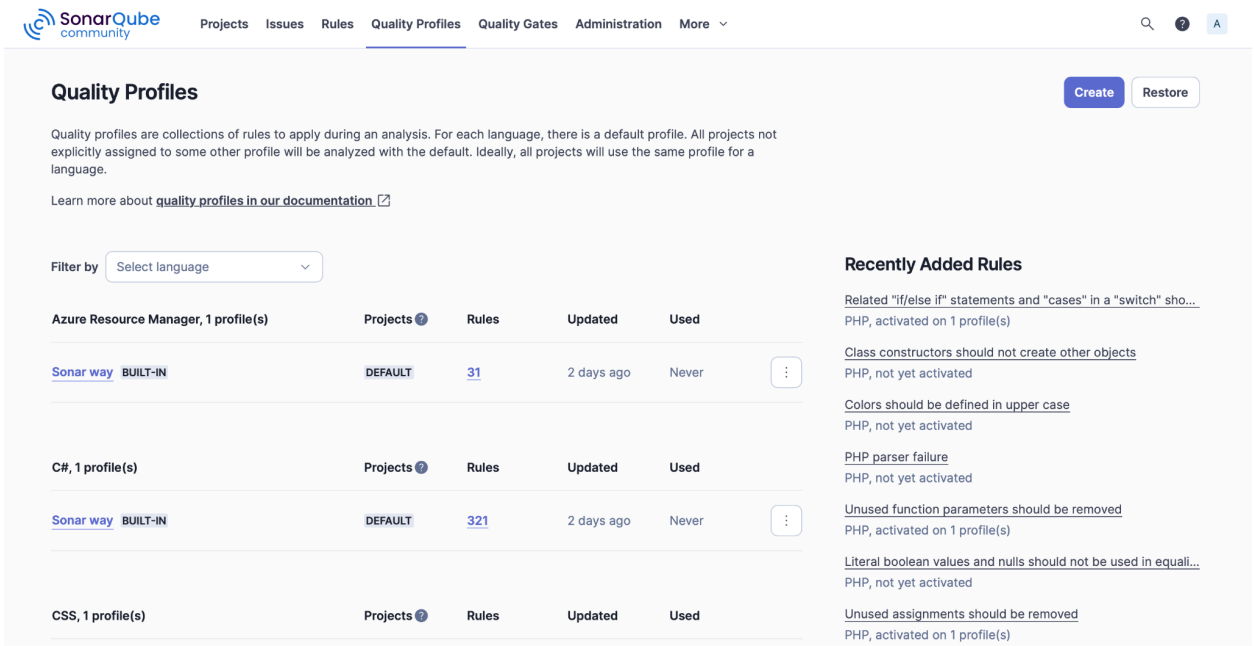
Quality Gate

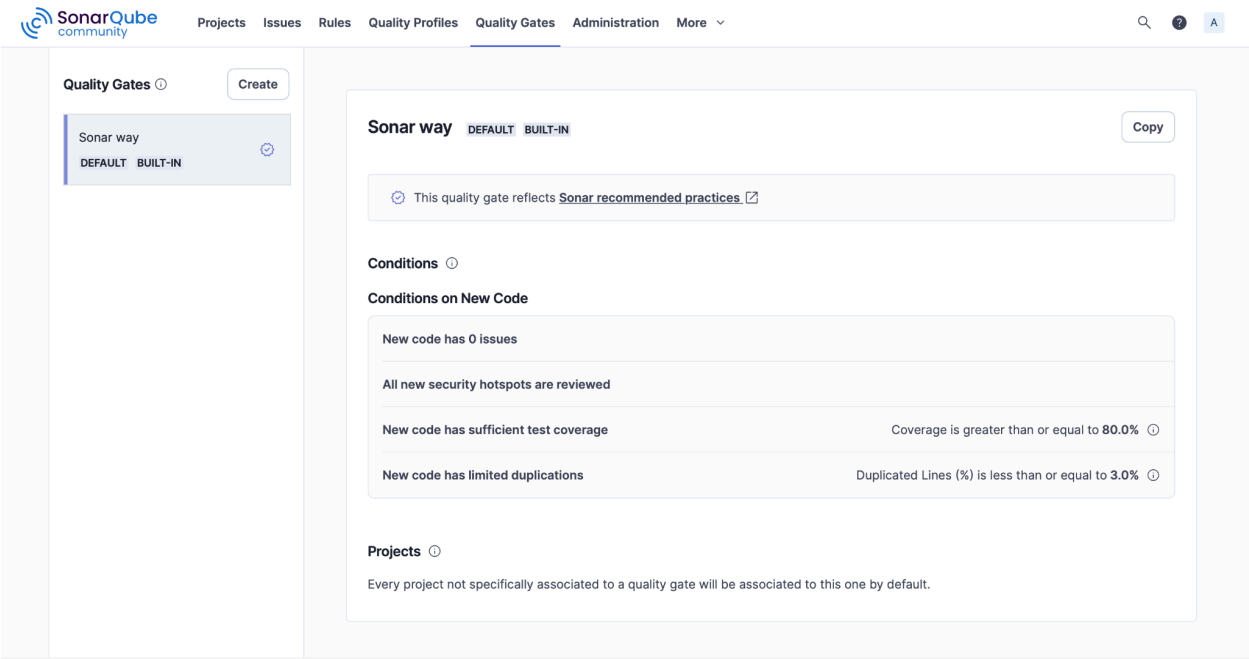
✓ Passed	1
× Failed	0


SonarQube community
Projects
Issues
Rules
Quality Profiles
Quality Gates
Administration
More
CheckStyle
Bind project
main
Overview
Issues
Security Hotspots
Code
Measures
Activity
Project Settings
Project Information
Project Overview
Security
Reliability
Maintainability
Security Review
Coverage
Duplications
Size
Complexity
Risk
See the data presented on this chart as a list
Color: Worse of Reliability Rating and Security Rating
Size: Lines of Code
A
B
C
D
E
Zoom: 100%
0.0%
20.0%
40.0%
60.0%
80.0%
Coverage
0
50min
1h 40min
2h 30min
3h 20min
4h 10min
5h


SonarQube community
Projects
Issues
Rules
Quality Profiles
Quality Gates
Administration
More
My Issues
All
Filters
Looking for Bugs, Vulnerabilities, or Code Smells? If your team prefers working with these types, change it in the settings
Software Quality
Security 2
Reliability 16
Maintainability 144
Severity
Blocker 1
High 28
Medium 63
Low 52
Info 18
Bulk Change
Select issues
Navigate to issue
162 issues
6d effort
CheckStyle / src/.../puppycrawl/tools/checkstyle/Checker.java
Split this "Monster Class" into smaller and more specialized ones to reduce its dependencies on other classes from 21 to the maximum authorized 20 or less.
Adaptability
Maintainability
Info
design
architecture
Open
Not assigned
L63 • 0min effort • 2 days ago
Remove useless curly braces around statement and then remove useless return keyword (sonar.java.source not set. Assuming 8 or greater.)
Intentionality
Maintainability
Low
java8
Open
Not assigned
L250 • 5min effort • 2 days ago
Refactor this method to reduce its Cognitive Complexity from 19 to the 15 allowed.
Adaptability
Maintainability
High
brain-overload
architecture
Open
Not assigned
L284 • 9min effort • 2 days ago
Catch Exception instead of Error.
Consistency
SonarQube™ technology is powered by SonarSource SA
Community Build • v25.10.0.114319 • • MQR MODE
LGPL v3
Community
Documentation
Plugins
Web API


SonarQube community
Projects
Issues
Rules
Quality Profiles
Quality Gates
Administration
More
Filters
Looking for Bugs, Vulnerabilities, or Code Smells? If your team prefers working with these types, change it in the settings
Language
Search for languages...
Java 695
TypeScript 492
JavaScript 475
C# 450
IPython Notebooks 351
5 shown Show More
Software quality
Security 262
Reliability 1.1k
Maintainability 2.5k
Search for rules...
Bulk Change
Select rules
Navigate to rule
4,061 rules
"!important" should not be used on "keyframes"
Intentionality
Reliability Medium
CSS
"$this" should not be used in a static context
Intentionality
Reliability Blocker
PHP
"&&" and "||" should be used
Intentionality
Maintainability Low
PHP suspicious
".equals()" should not be used to test the values of "Atomic" classes
Intentionality
Reliability Medium
Java multi-threading
".isEmpty" should be used to test for the emptiness of StringBuffers/Builders
Intentionality
Maintainability Low
Java performance clumsy
"<!DOCTYPE>" declarations should appear before "<html>" tags
Consistency
Reliability Medium
HTML user-experience
"<>" should not be used to test inequality
Consistency


SonarQube community
Projects
Issues
Rules
Quality Profiles
Quality Gates
Administration
More
Quality Profiles
Create
Restore
Quality profiles are collections of rules to apply during an analysis. For each language, there is a default profile. All projects not explicitly assigned to some other profile will be analyzed with the default. Ideally, all projects will use the same profile for a language.
Learn more about quality profiles in our documentation
Filter by
Select language
Azure Resource Manager, 1 profile(s)
Projects
Rules
Updated
Used
Sonar way BUILT-IN
DEFAULT
31
2 days ago
Never
C#, 1 profile(s)
Projects
Rules
Updated
Used
Sonar way BUILT-IN
DEFAULT
321
2 days ago
Never
CSS, 1 profile(s)
Projects
Rules
Updated
Used
Recently Added Rules
Related "if/else if" statements and "cases" in a "switch" sho...
PHP, activated on 1 profile(s)
Class constructors should not create other objects
PHP, not yet activated
Colors should be defined in upper case
PHP, not yet activated
PHP parser failure
PHP, not yet activated
Unused function parameters should be removed
PHP, activated on 1 profile(s)
Literal boolean values and nulls should not be used in equali...
PHP, not yet activated
Unused assignments should be removed
PHP, activated on 1 profile(s)



8. References

- Martin, R.C., *Clean Code*, 2008
- Fowler, M., *Refactoring*, 2018
- SonarQube Documentation, https://www.sonarqube.org
- CheckStyle Project Source Code, https://checkstyle.org